INSTITUTO POLITÉCNICO NACIONAL
Unidad Profesional Interdisciplinaria en Ingeniería y Tecnologías Avanzadas. UPIITA.

Códigos de Estado HTTP

Clasificación por categorías y ejemplos prácticos sobre el modelo TCP con funciones anónimas en JavaScript

Asignatura: Aplicaciones distribuidas

Alumno: CARDOSO GUTIERREZ MARIO DIDIER

Matrícula: 2024640195

Profesor: SIERRA ROMERO NOE

Fecha: 11 de noviembre de 2026

1. Introducción

El Protocolo de Transferencia de Hipertexto (HTTP) es el protocolo de la capa de aplicación sobre el que se construye la Web. Cada vez que un cliente realiza una petición, el servidor responde con un código de estado de tres dígitos que indica, de forma estandarizada, si la operación tuvo éxito, si el cliente debe actuar de otra manera o si ocurrió un error. Conocer estos códigos es fundamental para depurar aplicaciones, diseñar APIs REST y administrar servidores.

Este documento presenta los códigos de estado más importantes, clasificados por categorías de 1xx a 6xx, indicando su número, nombre en inglés, significado en español y un ejemplo práctico. Los ejemplos se implementan sobre el modelo TCP mediante funciones anónimas de JavaScript (Node.js) que escriben la respuesta directamente en el socket, de modo que se pueda observar el mensaje tal como viaja por la red.

Objetivos

- Identificar las clases de códigos de estado y el propósito de cada una.
- Relacionar cada código con un escenario práctico de uso.
- Comprender cómo se transmite una respuesta HTTP sobre una conexión TCP.

2. Marco teórico

2.1 HTTP sobre el modelo TCP/IP

En el modelo TCP/IP, HTTP/1.1 opera en la capa de aplicación y delega en TCP, en la capa de transporte, la entrega confiable y ordenada de los bytes. El flujo básico es el siguiente:

- El cliente abre una conexión TCP con el servidor mediante el acuerdo de tres vías (SYN, SYN-ACK, ACK), normalmente en el puerto 80 (o 443 si se usa TLS).
- El cliente envía una petición en texto plano.
- El servidor responde con un mensaje también en texto, que incluye el código de estado.
- Cualquiera de las partes cierra la conexión o la mantiene abierta para nuevas peticiones.

Capa	Protocolo	Función
Aplicación	HTTP	Define la petición, la respuesta y los códigos de estado.
Transporte	TCP	Conexión confiable, entrega ordenada y control de flujo.
Red	IP	Direccionamiento y enrutamiento de paquetes.
Enlace	Ethernet / Wi-Fi	Transmisión de tramas por el medio físico.

2.2 Estructura de una respuesta HTTP

Una respuesta HTTP/1.1 se compone de una línea de estado (versión, código y frase), encabezados, una línea en blanco y, opcionalmente, un cuerpo. Cada línea termina con los caracteres CRLF (`\r\n`).

```
HTTP/1.1 404 Not Found
Content-Type: text/html
Content-Length: 12

<h1>404</h1>
```

2.3 Clases de códigos de estado

El primer dígito del código define su clase. Si un cliente recibe un código que no conoce, debe tratarlo como el código genérico de su clase (por ejemplo, un 499 se interpreta como 400).

Clase	Categoría	Significado general
1xx	Informational	Petición recibida; el proceso continúa.
2xx	Success	La petición se recibió, entendió y aceptó.
3xx	Redirection	Se requiere una acción adicional para completar la petición.
4xx	Client Error	La petición es incorrecta o no puede cumplirse.
5xx	Server Error	El servidor falló al procesar una petición válida.
6xx	Global Failure (SIP)	No existe en HTTP; pertenece a SIP (ver sección 3.6).

2.4 Convención de los ejemplos

Los ejemplos usan el módulo `net` de Node.js, que ofrece sockets TCP puros (a diferencia del módulo `http`, que oculta el mensaje). Cada ejemplo del documento es una **función anónima** que recibe el socket de un cliente y se pasa como argumento a `net.createServer`:

```
const net = require('net');

net.createServer(
  /* función anónima de cada ejemplo */
).listen(8080);
```

Para probarlos basta ejecutar el servidor con `node servidor.js` y, en otra terminal, usar `curl -i http://localhost:8080/`. Se debe considerar lo siguiente:

- `socket.once('data', ...)` espera el primer segmento TCP recibido (simplificación: se asume que la petición llega completa en un solo segmento).
- `socket.write()` envía datos y mantiene abierta la conexión; `socket.end()` envía el último segmento y cierra la conexión TCP.
- El valor de `Content-Length` corresponde a los bytes del cuerpo que se envía en cada ejemplo.

3. Códigos de estado por categoría

3.1 Respuestas informativas (1xx) – Informational

Indican que la petición fue recibida y que el proceso continúa. Son respuestas provisionales: siempre van seguidas de una respuesta final y no llevan cuerpo.

Código	Nombre (inglés)	Significado (español)	Ejemplo práctico
100	Continue	El servidor recibió los encabezados y el cliente puede continuar enviando el cuerpo de la petición.	Un cliente va a subir un archivo de 200 MB y pregunta con «Expect: 100-continue» si el servidor lo aceptará.
101	Switching Protocols	El servidor acepta cambiar de protocolo, por ejemplo de HTTP a WebSocket.	El navegador abre un chat en tiempo real: la misma conexión TCP pasa a hablar WebSocket.
102	Processing	El servidor aceptó la petición y sigue procesándola; evita que el cliente agote su tiempo de espera (WebDAV, en desuso).	Copiar una carpeta enorme mediante WebDAV tarda minutos y el servidor avisa que continúa trabajando.
103	Early Hints	Pistas tempranas: el servidor adelanta encabezados Link para que el navegador precargue recursos mientras se prepara la respuesta final.	Una página consulta la base de datos durante 300 ms; mientras tanto el navegador ya descarga estilos.css.

Implementación con función anónima (TCP):

100 Continue

```
(socket) => {
  socket.once('data', () => {
    // Caso: el cliente envió «Expect: 100-continue» antes de subir un archivo
    socket.write('HTTP/1.1 100 Continue\r\n\r\n');
    socket.once('data', () => {
      socket.end('HTTP/1.1 201 Created\r\nContent-Length: 0\r\n\r\n');
    });
  });
}
```

101 Switching Protocols

```
(socket) => {
  socket.once('data', () => {
    // Caso: el cliente envió «Upgrade: websocket» (la conexión TCP sigue abierta)
    socket.write(
      'HTTP/1.1 101 Switching Protocols\r\n' +
      'Upgrade: websocket\r\n' +
      'Connection: Upgrade\r\n' +
      'Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=\r\n' +
      '\r\n'
    );
  });
}
```

102 Processing

```
(socket) => {
  socket.once('data', () => {
    // Caso: una operación WebDAV larga está en curso
    socket.write(
      'HTTP/1.1 102 Processing\r\n' +
      '\r\n'
    );
    socket.end(
      'HTTP/1.1 201 Created\r\n' +
      'Content-Length: 0\r\n' +
      '\r\n'
    );
  });
}
```

103 Early Hints

```
(socket) => {
  socket.once('data', () => {
    // Caso: se adelanta la precarga de una hoja de estilos
    socket.write(
      'HTTP/1.1 103 Early Hints\r\n' +
      'Link: </estilos.css>; rel=preload; as=style\r\n' +
      '\r\n'
    );
    socket.end(
      'HTTP/1.1 200 OK\r\n' +
      'Content-Type: text/html\r\n' +
      'Content-Length: 15\r\n' +
      '\r\n' +
      '<h1>Inicio</h1>'
    );
  });
}
```

3.2 Respuestas exitosas (2xx) – Success

Indican que la petición fue recibida, comprendida y aceptada correctamente.

Código	Nombre (inglés)	Significado (español)	Ejemplo práctico
200	OK	La petición se procesó correctamente y la respuesta contiene el recurso solicitado.	GET /index.html devuelve la página principal de un sitio.
201	Created	La petición tuvo éxito y se creó un nuevo recurso.	POST /usuarios registra a un alumno nuevo y devuelve su identificador.
202	Accepted	La petición fue aceptada, pero su procesamiento aún no termina.	El usuario pide un reporte anual en PDF; el servidor lo encola y responde de inmediato.
204	No Content	La petición tuvo éxito y no hay contenido que devolver.	DELETE /tareas/7 elimina la tarea y no necesita cuerpo de respuesta.
206	Partial Content	El servidor entrega solo una parte del recurso, según el encabezado Range.	Una descarga interrumpida se reanuda pidiendo únicamente los bytes faltantes (Range: bytes=0-4).

Implementación con función anónima (TCP):

200 OK

```
(socket) => {
  socket.once('data', () => {
    // Caso: GET /index.html
    socket.end(
      'HTTP/1.1 200 OK\r\n' +
      'Content-Type: text/html\r\n' +
      'Content-Length: 13\r\n' +
      '\r\n' +
      '<h1>Hola</h1>'
    );
  });
}
```

201 Created

```
(socket) => {
  socket.once('data', () => {
    // Caso: POST /usuarios con datos válidos
    socket.end(
      'HTTP/1.1 201 Created\r\n' +
      'Content-Type: application/json\r\n' +
      'Location: /usuarios/42\r\n' +
      'Content-Length: 9\r\n' +
      '\r\n' +
      '{"id":42}'
    );
  });
}
```

202 Accepted

```
(socket) => {
  socket.once('data', () => {
    // Caso: trabajo largo enviado a una cola
    socket.end(
      'HTTP/1.1 202 Accepted\r\n' +
      'Content-Type: application/json\r\n' +
      'Content-Length: 20\r\n' +
      '\r\n' +
      '{"estado":"en cola"}'
    );
  });
}
```

204 No Content

```
(socket) => {
  socket.once('data', () => {
    // Caso: DELETE /tareas/7
    socket.end(
      'HTTP/1.1 204 No Content\r\n' +
      '\r\n'
    );
  });
}
```

206 Partial Content

```
(socket) => {
  socket.once('data', () => {
    // Caso: petición con «Range: bytes=0-4»
    socket.end(
      'HTTP/1.1 206 Partial Content\r\n' +
      'Content-Range: bytes 0-4/1000\r\n' +
      'Content-Type: text/plain\r\n' +
      'Content-Length: 5\r\n' +
      '\r\n' +
      'ABCDE'
    );
  });
}
```

3.3 Redirecciones (3xx) – Redirection

Indican que el cliente debe realizar una acción adicional, normalmente seguir la URL indicada en el encabezado Location.

Código	Nombre (inglés)	Significado (español)	Ejemplo práctico
301	Moved Permanently	El recurso se movió de forma definitiva a otra URL indicada en Location.	Un sitio migra de http://ejemplo.com a https://www.ejemplo.com y redirige a todos los visitantes.
302	Found	Redirección temporal: el recurso se encuentra por ahora en otra URL.	Un usuario sin sesión que entra a /perfil es enviado temporalmente a /login.

Código	Nombre (inglés)	Significado (español)	Ejemplo práctico
303	See Other	El resultado se obtiene con una petición GET a otra URL.	Tras enviar un formulario con POST, el servidor redirige a /gracias para evitar reenvíos al recargar.
304	Not Modified	El recurso no cambió; el cliente puede usar su copia en caché.	El navegador envía If-None-Match con el ETag guardado y el servidor ahorra la transferencia del archivo.
307	Temporary Redirect	Redirección temporal que conserva el método y el cuerpo originales.	Una API en mantenimiento redirige un POST /pedidos a un servidor de respaldo sin convertirlo en GET.
308	Permanent Redirect	Redirección permanente que conserva el método y el cuerpo originales.	Una API mueve definitivamente /v1/pedidos a /v2/pedidos y los POST siguen funcionando.

Implementación con función anónima (TCP):

301 Moved Permanently

```
(socket) => {
  socket.once('data', () => {
    // Caso: cambio definitivo de dominio
    socket.end(
      'HTTP/1.1 301 Moved Permanently\r\n' +
      'Location: https://www.ejemplo.com/\r\n' +
      'Content-Length: 0\r\n' +
      '\r\n'
    );
  });
}
```

302 Found

```
(socket) => {
  socket.once('data', () => {
    // Caso: GET /perfil sin sesión iniciada
    socket.end(
      'HTTP/1.1 302 Found\r\n' +
      'Location: /login\r\n' +
      'Content-Length: 0\r\n' +
      '\r\n'
    );
  });
}
```

303 See Other

```
(socket) => {
  socket.once('data', () => {
    // Caso: POST /contacto procesado
    socket.end(
      'HTTP/1.1 303 See Other\r\n' +
      'Location: /gracias\r\n' +
      'Content-Length: 0\r\n' +
      '\r\n'
    );
  });
}
```

304 Not Modified

```
(socket) => {
  socket.once('data', () => {
    // Caso: el ETag del cliente sigue vigente
    socket.end(
      'HTTP/1.1 304 Not Modified\r\n' +
      'ETag: "abc123"\r\n' +
      '\r\n'
    );
  });
}
```

307 Temporary Redirect

```
(socket) => {
  socket.once('data', () => {
    // Caso: mantenimiento del servidor principal
    socket.end(
      'HTTP/1.1 307 Temporary Redirect\r\n' +
      'Location: https://respaldo.ejemplo.com/pedidos\r\n' +
      'Content-Length: 0\r\n' +
      '\r\n'
    );
  });
}
```

308 Permanent Redirect

```
(socket) => {
  socket.once('data', () => {
    // Caso: versión nueva de la API
    socket.end(
      'HTTP/1.1 308 Permanent Redirect\r\n' +
      'Location: /v2/pedidos\r\n' +
      'Content-Length: 0\r\n' +
      '\r\n'
    );
  });
}
```

3.4 Errores del cliente (4xx) – Client Error

Indican que la petición contiene un error o no puede cumplirse por causas atribuibles al cliente.

Código	Nombre (inglés)	Significado (español)	Ejemplo práctico
400	Bad Request	El servidor no puede procesar la petición por un error del cliente (sintaxis inválida).	Un formulario envía un JSON mal formado: falta cerrar una llave.
401	Unauthorized	Falta autenticación válida: el cliente debe identificarse.	Se consulta GET /notas sin enviar el token de sesión.
403	Forbidden	El servidor entendió la petición, pero se niega a ejecutarla por falta de permisos.	Un alumno autenticado intenta abrir el panel de administración.
404	Not Found	No se encontró el recurso solicitado.	GET /usuarios/99 cuando ese usuario no existe.
405	Method Not Allowed	El método HTTP no está permitido para ese recurso.	Se intenta DELETE sobre /reportes, que solo acepta GET.
408	Request Timeout	El servidor esperó demasiado tiempo la petición completa del cliente.	Un cliente abre la conexión TCP y no envía nada durante 5 segundos.
409	Conflict	La petición choca con el estado actual del recurso.	Registrar un usuario con un correo electrónico que ya existe.
410	Gone	El recurso existió, pero fue eliminado de forma permanente.	Una promoción terminó y su página fue retirada definitivamente.
413	Content Too Large (antes Payload Too Large)	El cuerpo de la petición excede el límite que acepta el servidor.	Se sube una foto de 20 MB a un formulario cuyo límite es 5 MB.
415	Unsupported Media Type	El formato del cuerpo no es soportado por el servidor.	Se envía XML a una API que solo acepta application/json.
422	Unprocessable Content (antes Unprocessable Entity)	La sintaxis es correcta, pero los datos no pasan las validaciones semánticas.	Un JSON válido trae una edad de −5 años.
429	Too Many Requests	El cliente superó el límite de peticiones permitido (rate limiting).	Un script consulta la API 200 veces por minuto cuando el límite es 60.

Implementación con función anónima (TCP):

400 Bad Request

```
(socket) => {
  socket.once('data', () => {
    // Caso: el cuerpo JSON no se puede interpretar
    socket.end(
      'HTTP/1.1 400 Bad Request\r\n' +
      'Content-Type: application/json\r\n' +
      'Content-Length: 25\r\n' +
      '\r\n' +
      '{"error":"JSON invalido"}'
    );
  });
}
```

401 Unauthorized

```
(socket) => {
  socket.once('data', () => {
    // Caso: no llegó el encabezado Authorization
    socket.end(
      'HTTP/1.1 401 Unauthorized\r\n' +
      'WWW-Authenticate: Bearer\r\n' +
      'Content-Type: application/json\r\n' +
      'Content-Length: 27\r\n' +
      '\r\n' +
      '{"error":"Token requerido"}'
    );
  });
}
```

403 Forbidden

```
(socket) => {
  socket.once('data', () => {
    // Caso: usuario válido sin rol de administrador
    socket.end(
      'HTTP/1.1 403 Forbidden\r\n' +
      'Content-Type: application/json\r\n' +
      'Content-Length: 27\r\n' +
      '\r\n' +
      '{"error":"Acceso denegado"}'
    );
  });
}
```

404 Not Found

```
(socket) => {
  socket.once('data', () => {
    // Caso: GET /usuarios/99 sin registro en la base de datos
    socket.end(
      'HTTP/1.1 404 Not Found\r\n' +
      'Content-Type: text/html\r\n' +
      'Content-Length: 12\r\n' +
      '\r\n' +
      '<h1>404</h1>'
    );
  });
}
```

405 Method Not Allowed

```
(socket) => {
  socket.once('data', () => {
    // Caso: DELETE /reportes
    socket.end(
      'HTTP/1.1 405 Method Not Allowed\r\n' +
      'Allow: GET, HEAD\r\n' +
      'Content-Length: 0\r\n' +
      '\r\n'
    );
  });
}
```

408 Request Timeout

```
(socket) => {
  socket.setTimeout(5000, () => {
    // Caso: pasaron 5 s y el cliente no envió ninguna petición
    socket.end(
      'HTTP/1.1 408 Request Timeout\r\n' +
      'Connection: close\r\n' +
      'Content-Length: 0\r\n' +
      '\r\n'
    );
  });
}
```

409 Conflict

```
(socket) => {
  socket.once('data', () => {
    // Caso: POST /usuarios con correo repetido
    socket.end(
      'HTTP/1.1 409 Conflict\r\n' +
      'Content-Type: application/json\r\n' +
      'Content-Length: 32\r\n' +
      '\r\n' +
      '{"error":"Correo ya registrado"}'
    );
  });
}
```

410 Gone

```
(socket) => {
  socket.once('data', () => {
    // Caso: GET /promo-2025 (página retirada)
    socket.end(
      'HTTP/1.1 410 Gone\r\n' +
      'Content-Length: 0\r\n' +
      '\r\n'
    );
  });
}
```

413 Content Too Large

```
(socket) => {
  socket.once('data', () => {
    // Caso: Content-Length mayor a 5 MB
    socket.end(
      'HTTP/1.1 413 Content Too Large\r\n' +
      'Content-Type: application/json\r\n' +
      'Content-Length: 23\r\n' +
      '\r\n' +
      '{"error":"Maximo 5 MB"}'
    );
  });
}
```

415 Unsupported Media Type

```
(socket) => {
  socket.once('data', () => {
    // Caso: Content-Type: application/xml
    socket.end(
      'HTTP/1.1 415 Unsupported Media Type\r\n' +
      'Content-Type: application/json\r\n' +
      'Content-Length: 32\r\n' +
      '\r\n' +
      '{"error":"Use application/json"}'
    );
  });
}
```

422 Unprocessable Content

```
(socket) => {
  socket.once('data', () => {
    // Caso: JSON bien formado con datos inválidos
    socket.end(
      'HTTP/1.1 422 Unprocessable Content\r\n' +
      'Content-Type: application/json\r\n' +
      'Content-Length: 25\r\n' +
      '\r\n' +
      '{"error":"edad invalida"}'
    );
  });
}
```

429 Too Many Requests

```
(socket) => {
  socket.once('data', () => {
    // Caso: cuota por minuto agotada
    socket.end(
      'HTTP/1.1 429 Too Many Requests\r\n' +
      'Retry-After: 60\r\n' +
      'Content-Length: 0\r\n' +
      '\r\n'
    );
  });
}
```

3.5 Errores del servidor (5xx) – Server Error

Indican que el servidor falló al intentar cumplir una petición que, en principio, era válida.

Código	Nombre (inglés)	Significado (español)	Ejemplo práctico
500	Internal Server Error	Error inesperado del servidor al procesar la petición.	Una excepción no controlada interrumpe el código que atiende la ruta.
501	Not Implemented	El servidor no reconoce o no implementa el método solicitado.	Se envía PATCH a un servidor que solo implementa GET y POST.
502	Bad Gateway	Un servidor que actúa como puerta de enlace recibió una respuesta inválida del servidor de origen.	Un proxy inverso (Nginx) no recibe una respuesta válida de la aplicación en Node.js.

Código	Nombre (inglés)	Significado (español)	Ejemplo práctico
503	Service Unavailable	El servidor no puede atender la petición por sobrecarga o mantenimiento temporal.	En semana de inscripciones el servidor se satura y pide reintentar en 2 minutos.
504	Gateway Timeout	La puerta de enlace no recibió respuesta a tiempo del servidor de origen.	Un proxy espera 30 s a una base de datos lenta y se rinde.
505	HTTP Version Not Supported	El servidor no soporta la versión de HTTP usada en la petición.	Un cliente envía una petición con la versión HTTP/9.9.

Implementación con función anónima (TCP):

500 Internal Server Error

```
(socket) => {
  socket.once('data', () => {
    // Caso: se lanzó una excepción no controlada
    socket.end(
      'HTTP/1.1 500 Internal Server Error\r\n' +
      'Content-Type: application/json\r\n' +
      'Content-Length: 25\r\n' +
      '\r\n' +
      '{"error":"Error interno"}'
    );
  });
}
```

501 Not Implemented

```
(socket) => {
  socket.once('data', () => {
    // Caso: método PATCH sin soporte
    socket.end(
      'HTTP/1.1 501 Not Implemented\r\n' +
      'Content-Length: 0\r\n' +
      '\r\n'
    );
  });
}
```

502 Bad Gateway

```
(socket) => {
  socket.once('data', () => {
    // Caso: el servidor de origen devolvió datos corruptos
    socket.end(
      'HTTP/1.1 502 Bad Gateway\r\n' +
      'Content-Length: 0\r\n' +
      '\r\n'
    );
  });
}
```

503 Service Unavailable

```
(socket) => {
  socket.once('data', () => {
    // Caso: servidor saturado
    socket.end(
      'HTTP/1.1 503 Service Unavailable\r\n' +
      'Retry-After: 120\r\n' +
      'Content-Length: 0\r\n' +
      '\r\n'
    );
  });
}
```

504 Gateway Timeout

```
(socket) => {
  socket.once('data', () => {
    // Caso: el servidor de origen no respondió en 30 s
    socket.end(
      'HTTP/1.1 504 Gateway Timeout\r\n' +
      'Content-Length: 0\r\n' +
      '\r\n'
    );
  });
}
```

505 HTTP Version Not Supported

```
(socket) => {
  socket.once('data', () => {
    // Caso: versión de protocolo desconocida
    socket.end(
      'HTTP/1.1 505 HTTP Version Not Supported\r\n' +
      'Content-Length: 0\r\n' +
      '\r\n'
    );
  });
}
```

3.6 Categoría 6xx (no pertenece a HTTP)

El estándar HTTP (RFC 9110) solo define las clases 1xx a 5xx, y los valores fuera de 100–599 no son códigos HTTP válidos. La clase 6xx existe en otro protocolo de aplicación que también puede viajar sobre TCP: SIP (Session Initiation Protocol, RFC 3261), usado en telefonía IP, donde significa «fallo global» (el error no depende del servidor consultado). Se incluyen aquí por completitud, con el mismo enfoque de ejemplos sobre TCP (puerto 5060).

Código	Nombre (inglés)	Significado (español)	Ejemplo práctico
600	Busy Everywhere	Todos los destinos del usuario están ocupados.	Una llamada VoIP a una persona que está en otra llamada en todos sus dispositivos.
603	Decline	El usuario rechaza la llamada de forma definitiva.	El destinatario presiona «rechazar» en su teléfono IP.

Código	Nombre (inglés)	Significado (español)	Ejemplo práctico
604	Does Not Exist Anywhere	El usuario no existe en ningún servidor del dominio.	Se llama a la extensión 9999, que nadie tiene registrada.

Implementación con función anónima (TCP):

600 Busy Everywhere

```
(socket) => {
  socket.once('data', () => {
    // Caso: el usuario está ocupado en todos sus equipos
    socket.end(
      'SIP/2.0 600 Busy Everywhere\r\n' +
      'Content-Length: 0\r\n' +
      '\r\n'
    );
  });
}
```

603 Decline

```
(socket) => {
  socket.once('data', () => {
    // Caso: el usuario rechazó la llamada
    socket.end(
      'SIP/2.0 603 Decline\r\n' +
      'Content-Length: 0\r\n' +
      '\r\n'
    );
  });
}
```

604 Does Not Exist Anywhere

```
(socket) => {
  socket.once('data', () => {
    // Caso: extensión inexistente en todo el dominio
    socket.end(
      'SIP/2.0 604 Does Not Exist Anywhere\r\n' +
      'Content-Length: 0\r\n' +
      '\r\n'
    );
  });
}
```

4. Conclusiones

Los códigos de estado son el lenguaje común con el que cliente y servidor comunican el resultado de cada petición. Las cinco clases definidas por HTTP (1xx a 5xx) permiten reaccionar de forma automática: continuar, aceptar el resultado, seguir una redirección, corregir la petición o reintentar más tarde.

Al implementarlos directamente sobre sockets TCP se observa que una respuesta HTTP no es más que texto con un formato preciso, transportado por una conexión confiable. Finalmente, la clase 6xx no forma parte de HTTP, sino de protocolos como SIP, por lo que conviene no confundirla con los códigos de estado de la Web.

5. Referencias

Fielding, R., Nottingham, M., & Reschke, J. (2022). RFC 9110: HTTP Semantics. Internet Engineering Task Force (IETF).

Fielding, R., Nottingham, M., & Reschke, J. (2022). RFC 9112: HTTP/1.1. Internet Engineering Task Force (IETF).

Postel, J. (1981). RFC 793: Transmission Control Protocol. Internet Engineering Task Force (IETF).

Rosenberg, J., et al. (2002). RFC 3261: SIP: Session Initiation Protocol. Internet Engineering Task Force (IETF).

Fette, I., & Melnikov, A. (2011). RFC 6455: The WebSocket Protocol. Internet Engineering Task Force (IETF).

Oku, K. (2017). RFC 8297: An HTTP Status Code for Indicating Hints. Internet Engineering Task Force (IETF).

Internet Assigned Numbers Authority. (s. f.). Hypertext Transfer Protocol (HTTP) Status Code Registry. https://www.iana.org/assignments/http-status-codes

OpenJS Foundation. (s. f.). Node.js documentation: Net. https://nodejs.org/api/net.html